## קטע קוד A

```
 1. class Expression; // forward declaration
 2.
 3. class BaseExpression {
 4.   friend class Expression;
 5.   mutable size_t refCount = 0;
 6.   void incRefCount() const { ++refCount; }
 7.   size_t decRefCount() const { return --refCount; }
 8. protected:
 9.   mutable Expression* parent = nullptr;
10. public:
11.   BaseExpression() = default;
12.   BaseExpression(const BaseExpression&) = delete;
13.   virtual ~BaseExpression() {}
14.   virtual double eval() const = 0;
15.   virtual ostream& print(ostream& out) const = 0;
16.   virtual void setParent(Expression* e) const { parent = e; }
17.   friend ostream& operator<<(ostream& o, const BaseExpression& e) {
18.     return e.print(out);
19.   }
20. };
21.
22. class Number: public BaseExpression {
23.   double value;
24. public:
25.   Number(double d) : value(d) {}
26.   double eval() const override { return value; }
27.   ostream& print(ostream& out) const override {
28.     return out << value;
29.   }
30. };
31.
32. class Var: public BaseExpression {
33.   const std::string name;
34. public:
35.   Var(const std::string& name) : name(name) {}
36.   double eval() const override; // implementation in line 76
37.   ostream& print(ostream& out) const override {
38.     return out << name;
39.   }
40. };
41.
42. class Expression {
43.   const BaseExpression* e;
44.   std::map<std::string, double> vars;
45.   mutable Expression* parent = nullptr;
46. public:
47.   Expression(const BaseExpression* be): e(be) {
48.     e->setParent(this);
49.     e->incRefCount();
50.   }
51.   Expression(double d): Expression(new Number(d)) {}
52.   Expression(const std::string& s): Expression(new Var(s)) {}
53.   Expression(const char* s): Expression(std::string(s)) {}
54.   Expression(const Expression& other): Expression(other.e) {
55.     vars = other.vars;
56.   }
```

```
~Expression() { if(e->decRefCount() == 0) delete e; }
double eval() const { return e->eval(); }
void setParent(Expression* e) const { parent = e; }
void setVar(const std::string& var_name, double value) {
  vars[var_name] = value;
}
void removeVar(const std::string& var_name) {
  vars.erase(var_name); // removes an entry from the map
}
double getValue(const std::string& var_name) const {
  auto itr = vars.find(var_name);
  if(itr != vars.end()) return itr->second;
  return parent == nullptr? 0 : parent->getValue(var_name);
}
friend ostream& operator<<(ostream& out, const Expression& e) {
  return out << *e.e;
}
};

double Var::eval() const {
  return parent != nullptr ? parent->getValue(name) : 0;
}

template<char Sign>
class BinaryExpression : public BaseExpression {
  Expression e1, e2;
public:
BinaryExpression (Expression a, Expression b) : e1(a), e2(b) {}
  virtual double evalImpl(double d1, double d2) const = 0;
double eval() const override {
  return evalImpl(e1.eval(), e2.eval());
}
ostream& print(ostream& out) const override {
  return out << '(' << e1 << ' ' << Sign << ' ' << e2 << ')';
}
void setParent(Expression* e) const override {
  e1.setParent(e); e2.setParent(e);
}
};

struct Sum : public BinaryExpression<'+'> {
  using BinaryExpression::BinaryExpression;
double evalImpl(double d1, double d2) const override {
  return d1 + d2;
}
};

struct Mult: public BinaryExpression<'*'> {
  using BinaryExpression::BinaryExpression;
double evalImpl(double d1, double d2) const override {
  return d1 * d2;
}
};

template<typename ExpressionType>
Expression E(Expression e1, Expression e2) {
  return new ExpressionType(e1, e2);
}
```

להלן מספר main אפשריים:

## A1 main

```
int main() {
  auto e1 = E<Sum>( E<Mult>("x", "x"), "x" );   ((x·x)+x)
  e1.setVar("x", 5);
  cout << e1.eval() << endl;  30
  e1.setVar("x", 1);
  cout << e1.eval() << endl;  2
  e1.removeVar("x");
  cout << e1.eval() << endl;  0
}
```

## A2 main

```
int main() {
  auto e1 = E<Sum>( "x", 5 );  x+5
  e1.setVar("x", 5);
  cout << e1 << " = " << e1.eval() << std;  x+5=10
  auto e2 = E<Mult>( e1, "x" );  (x+5)·x
  e2.setVar("x", 20);
  // e1.removeVar("x");
  cout << e2 << " = " << e2.eval() << std;  (x+5)·x=200
}
```

## A3 main

```
int main() {
  auto e1 = E<Sum>( "x", 5 );  x+5
  e1.setVar("x", 5);  x=5
  cout << e1 << " = " << e1.eval() << endl;  10
  e1.removeVar("x");  מתקיים
  cout << e1 << " = " << e1.eval() << endl;  x+5=5
  auto e2 = E<Mult>( e1, "x" );  (x+5)·x   Parent e2 ל'
  e2.setVar("x", 10);  x=10
  cout << e2 << " = " << e2.eval() << endl;  (x+5)x=150
  e1.setVar("x", 5);  → משנה ל e1
  cout << e2 << " = " << e2.eval() << endl;
}
ולא לפי
```

## A4 main

```
int main() {
  auto e = new Sum(new Mult(2, 4), 3);   (2·4)+3
  cout << *e << " = " << e->eval() << endl;
  delete e;
}
```

## A5 main

```
int main() {
  Expression e = new Sum(new Mult(2, "x"), 3);
  e.setVar("x", 4);
  cout << e << " = " << e.eval() << endl;
}
```

## קטע קוד B – לשאלות הבונוס

```
struct Bar {
  Bar() = default;
  Bar(const Bar&) {
    cout << "Bar copy ctor" << endl;
  }
  Bar(Bar&&) {
    cout << "Bar move ctor" << endl;
  }
};

class Foo {
  Bar b_;
public:
  Foo(Bar&& b = Bar()): b_(b) {}
  Foo(Foo&& other): b_(std::move(other.b_)) {
    cout << "Foo move ctor" << endl;
  }
};

int main() {
  // Foo f1(Bar{});
  // Bar b;
  // Foo f2(b);
  // Foo f3;
  // Foo f4(f3);
  // Foo f5(std::move(f3));
}
```

**בהצלחה !**